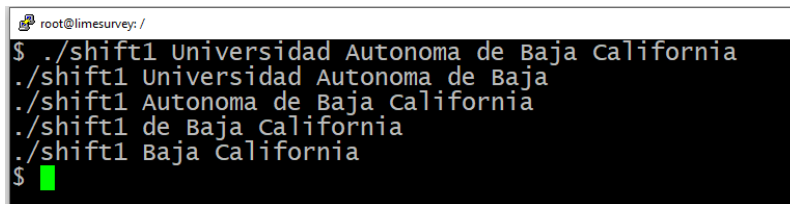


Shift

El comando shift rota a la izquierda todos los parámetros de la línea de comando a partir de la posición 0, es decir, el parámetro \$2 se desplaza hacia el \$1, el \$3 al \$2 etc. El valor anterior de \$1 se descarta mientras que la posición \$0 permanece porque es el nombre del script.

Ejemplo 1 shift

```
#!/bin/sh
#Shift1
echo $0 $1 $2 $3 $4
shift
echo $0 $1 $2 $3 $4
shift
echo $0 $1 $2 $3 $4
shift
echo $0 $1 $2 $3 $4
```

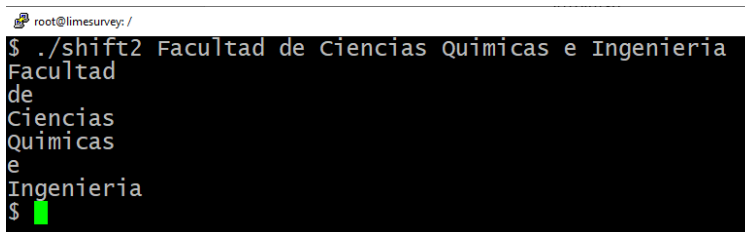


```
root@limesurvey: /
$ ./shift1 Universidad Autonoma de Baja California
./shift1 Universidad Autonoma de Baja
./shift1 Autonoma de Baja California
./shift1 de Baja California
./shift1 Baja California
$
```

Ejemplo 2 shift con ciclo while

```
#!/bin/sh
#shift2

while [ "$1" != "" ]; do
    echo $1
    shift
done
```



```
root@limesurvey: /
$ ./shift2 Facultad de Ciencias Quimicas e Ingenieria
Facultad
de
Ciencias
Quimicas
e
Ingenieria
$
```

Procesamiento Selectivo

El procesamiento selectivo permite la ejecución de un conjunto de comandos de shell si la evaluación de una expresión es verdadera. El shell BASH tiene un conjunto de estructuras condicionales de control estructuras, similares a las que se encuentran en los lenguajes de programación, esta son test, if y case.

Comando **test** o **[]**.

En la práctica la mayoría de los scripts hacen extensivo el uso de los corchetes o **test** para probar expresiones. Se ejecutará una acción si la expresión de prueba es verdadera.

Con []

```
if [ expresión ]; then
    operaciones
fi
```

Con test

```
if test expresión; then
    operaciones
fi
```

Se ejecutará una acción si la expresión de prueba es verdadera. Si no se ejecutarán las operaciones de la parte falsa (**else**).

Con []

```
if [ expresión ]; then
    operaciones
else
    operaciones
fi
```

Con test

```
if test expresión; then
    operaciones
else
    operaciones
fi
```

La siguiente estructura del **if** permite anidar estructuras **if**, para permitir la selección de múltiples alternativas. Utiliza el comando **elif**, el cual prueba la variable de nuevo si el primer **if** no fue verdadero. Si ninguna de las pruebas es verdadero, entonces se ejecuta la parte falsa del **elif**.

```
if [ expresión ]; then
    operaciones
elseif [ expresión ]; then
    operaciones
else
    operaciones
fi
```

Nota: Entre la expresión y los corchetes siempre debe dejar un espacio.
Puede sustituir el uso de corchetes por el comando **test**.

Comparación de cadenas.

= Verdadero si las cadenas son iguales.

!= Verdadero si las cadenas son diferentes.
-n cadena Verdadero si la cadena no es null. //NO FUNCIONA A MENOS QUE ESTE ENTRE COMILLAS "\$1"
-z cadena Verdadero si la cadena es null (una cadena vacía).

Comparación de números

expresión1 -eq expresión2 Verdadero si las expresiones son iguales.
expresión1 -ne expresión2 Verdadero si las expresiones no son iguales.
expresión1 -gt expresión2 Verdadero si la expresión1 es mayor que expresión2.
expresión1 -ge expresión2 Verdadero si la expresión1 es mayor o igual que expresión2.
expresión1 -lt expresión2 Verdadero si la expresión1 es menor que expresión2.
expresión1 -le expresión2 Verdadero si la expresión1 es menor o igual que expresión2.
!expresión El símbolo ! niega la expresión y regresa verdadero si la expresión es falsa.

Operadores Lógicos

! Not
-a And
-o Or

Operadores de archivos

-d archivo Verdadero si el archivo es un directorio.
-e archivo Verdadero si el archivo existe.
-f archivo Verdadero si el archivo normal.
-r archivo Verdadero si el archivo se puede leer.
-s archivo Verdadero si el archivo tiene longitud mayor de 0 bytes.
-w archivo Verdadero si el archivo se puede escribir.
-x archivo Verdadero si el archivo se puede ejecutar.

Ejemplo 1

#Recibe dos parámetros numéricos de la línea de comando y determina cual de los dos es mayor.
Utiliza como comando de prueba los corchetes.

```
#!/bin/bash
clear
if [ $1 -gt $2 ]; then
    echo a mayor que b

    elif [ $1 -eq $2 ]; then
        echo a igual b
    else
        echo b mayor que a
    fi
```

En el ejemplo anterior se emplea el comando [] para la prueba de condiciones, ahora veamos el mismo ejemplo trabajando con el comando test.

Ejemplo 2

#Recibe dos parámetros numéricos de la línea de comando y determina cuál de los dos es mayor.
Utiliza como comando de prueba test.

```
#!/bin/bash
clear
if test $1 -gt $2
then
    echo a mayor que b

elif test $1 -eq $2 ; then
    echo a igual b
else
    echo b mayor que a
fi
```

Ejemplo 3

Comparacion de cadenas ifEjemplo3
El programa revisa si recibió un parametro de la linea de comando. Si no lo recibió, pide una ciudad. Y la compara con la cadena Mexicali

```
#!/bin/bash

miciudad="mexicali"

if [ -z $1 ]; then
    echo "De donde eres?"
    read tuciudad
else
    tuciudad=$1
fi

if [ $miciudad = $tuciudad ]; then
    echo "Somos paisanos"
else
    echo "Hola"
fi
```

Las operadores lógicos && y ||.

orden && orden

Iniciando de izquierda a derecha, cada comando se ejecutará siempre y cuando el comando anterior se haya ejecutado con éxito. Si todas las ordenes se ejecutan con éxito, el enunciado AND, considerándose como un todo regresará un valor verdadero. En el momento que uno de los comandos no se ejecute correctamente, entonces se las demás ya no serán ejecutadas y la orden AND tendrá un valor falso.

orden || orden

Los comandos se ejecutan comenzando de izquierda a derecha, se van ejecutando uno por uno, hasta que uno regresa verdadero. Entonces la ejecución de la lista de comandos se detiene y la orden OR regresará un valor verdadero. Si ninguno de los comandos de la lista OR regresa un valor verdadero, entonces la orden OR regresará un valor falso.

Operador Lógico AND

Este programa prueba si existe el primer archivo, si existe envía el mensaje, si la ejecución del comando echo

se realiza con éxito, entonces se prueba el segundo archivo y si este también existe entonces se envía el mensaje. Si todas las instrucciones en el if se realizaron con éxito, entonces se ejecuta la parte verdadera del if.

Si alguna no se ejecuta con éxito, entonces se suspende la ejecución de las demás y se ejecuta la parte falsa #del if.

```
#And
#Este programa prueba si existen los 3 archivos
if [ $# -ge 3 ]; then
    if [ -e $1 ] && [ -e $2 ]; then
        echo "$1, $2 y $3 existen"
    else
        echo "alguno de los archivos no existe"
    fi
else
    echo "No hay parametros"
fi
```

Operador Lógico OR

#Prueba si existe archivo1, si no existe intenta con archivo2 y luego con archivo3. Si ninguno existe Or regresa un valor #falso y se ejecuta la parte falsa del if. Si al probar la existencia de uno de los archivos el resultado es verdadero, entonces #las siguientes instrucciones a la derecha ya no serán ejecutadas y la orden OR tomará un valor de verdadero, por lo que #se ejecutará la parte verdadera del if.

```
#!/bin/sh
echo " Hay $# parametros "
if [ $# -ge 3 ]; then
    if [ -e $1 ] || [ -e $2 ] || [ -e $3 ]; then
        echo "Al menos uno de los archivos existe"
    else
        echo "Ninguno de los archivos no existe"
    fi
else
    echo "No escribiste suficientes parametros"
fi
```

case

La estructura **case** permite seleccionar entre alternativas mediante la comparación de una variable con varios patrones posibles, asociados a un conjunto de instrucciones que se ejecutarán cuando se encuentre una coincidencia. La estructura inicia con la palabra reservada **case**, luego una variable y la palabra **in**. Enseguida se escriben una lista de patrones, al final de cada patrón se escribe un paréntesis derecho. Después del paréntesis se listan los comandos asociados al patrón, para marcar el fin de este bloque de comandos se escribe dos veces el punto y coma. Finalmente, **case** se cierra con la palabra reservada **esac**.

Si se desea realizar operaciones cuando no se encuentre correspondencia entre las variables y ninguno de los patrones, entonces puede utilizarse un caso default. El caso default se indica agregando a la lista de patrones de comparación, el asterisco como si fuera otro caso más. La sintaxis es la siguiente:

```
case variable in
    patrón1)  instrucciones
            ;;
    patrón2)  instrucciones
            ;;
    patrónN)  instrucciones
            ;;
    *) instrucciones
            ;;
esac
```

Dentro de un patrón se puede incluir cualquier caracter especial. Los caracteres especiales del shell son el asterisco, los corchetes, el signo de interrogación y el pipe. En el case, puede aplicar el uso de operadores lógicos para asociar más de un patrón un bloque de instrucciones.

Sintaxis

```
case variable in
    patron1 | patron2)  instrucciones
            ;;
    patron3)            instrucciones
            ;;
    *)                  instrucciones
esac
```

Ejemplo

```
#!/bin/sh
#Ejemplo de case.
# Recibe un numero de la linea de comando y determina que mes es.

case $1 in
  01 | 1 | uno) echo "el mes es enero";;
  02 | 2 | [Dd][Oo][Ss]) echo "el mes es febrero";;
  03 | 3) echo "el mes es marzo";;
  04 | 4) echo "el mes es abril";;
  05 | 5) echo "el mes es mayo";;
  06 | 6) echo "el mes es junio";;
  07 | 1) echo "el mes es julio";;
  08 | 1) echo "el mes es agosto";;
  09 | 1) echo "el mes es septiembre";;
  10) echo "el mes es octubre";;
  11) echo "el mes es noviembre";;
  12) echo "el mes es diciembre";;
  *) echo -e "\a \0344 Error";;
esac
```

En el caso 1 podemos observar que, aplicando los operadores lógicos se pueden asociar más de un patrón con un bloque de instrucciones. El usuario puede escribir desde la línea de comando uno, 1 o 01 y de cualquier modo la respuesta del programa será “el mes es enero”.

En el caso 2 se hace uso de los corchetes para darle mayor flexibilidad a la búsqueda de patrones. Mientras que en el caso 1, solo se puede identificar la palabra uno escrita con minúsculas, en el caso 2 se podrá reconocer sin importar si está escrita en mayúsculas, minúsculas, o en una combinación de ambas.

Ciclo for-in

La estructura **for-in** está diseñada para hacer referencia de forma secuencial a una lista de valores. Requiere de dos operandos, estos son: una variable y una lista de valores. Cada elemento de la lista se asigna uno a uno a la variable de la estructura **for-in**. Cuando se llega al final de la lista el bucle se detiene, al igual que en el ciclo while, el cuerpo de ciclo inicia con la palabra reservada **do** y termina con **done**. La sintaxis del ciclo **for-in** es la siguiente:

```
for variable in lista_de_valores
do
    instrucciones
done
```

Ejemplo 1

```
#Imprime hola seguido por los nombres de la lista, cada uno en un renglón diferente.
#!/bin/sh
for nombre in Coco Tina Janis Maggie Honey Jason Pepper
do
    echo "hola $nombre"
done
```

Ejemplo 2

#Imprime en cada renglón hola y enseguida uno de los nombres de la lista, los valores para la lista se toman de # la línea de comando.

```
#!/bin/bash
```

```
for nombre in $*
do
    echo "Hola $nombre"
done
```

Ejemplo 3

El ciclo imprime los datos nombre y plan de cada una de las personas en la lista.

Los valores para el ciclo se obtienen de la ejecución del comando cut sobre un archivo creado previamente,

#llamado datos. En cada iteración espera a que se presione una tecla para mostrar los datos de la persona

siguiente.

```
#!/bin/bash
```

```
lista=`cut -d':' -f1 datos`
for nombre in $lista
do
    nom=`finger -m $nombre | grep ^Login | cut -c32-65`
    plan=`finger -m $nombre | tail -1`
    echo "$nombre:$nom:$plan"
    read
done
```

Ciclo while

Este ciclo es equivalente al ciclo for. Un ciclo while empieza con la palabra reservada **while** y va seguido por una orden que, usualmente, efectúa una prueba. Luego se escribe la palabra do y a continuación se escriben todas las instrucciones que forman el lazo, al final se especifica mediante la palabra reservada **done**. Su sintaxis es:

```
while orden
do
    instrucciones
done
```

```
while (expresión)
    comandos
end
```


Donde expresión es una expresión que se evaluará antes de ejecutar cualquiera de los comandos siguientes. Si la expresión es verdadera, los comandos se ejecutan hasta llegar a la sentencia **end**, y después se vuelve a evaluar la expresión. Si la expresión es falsa, se inicia el comando que sigue a **end**.

```
#while1
#Este programa pregunta al usuario un password hasta que escriba la palabra clave.
#!/bin/bash

echo " Trata de adivinar mi password"
read intento
cont=0

while [ $intento != "soporte123" ];
do
    echo " Ese no es, intentalo otra vez"
    read intento
    cont=`expr $cont + 1`
done
echo "Fueron $cont intentos"
```

Ciclo until

Este ciclo es similar al ciclo while, pero la condición se prueba al revés. En otras palabras, el ciclo continúa hasta que la condición se hace verdadera, no mientras la condición es verdad.

```
until condición
do
    operaciones
done
```

Ejemplo:

```
#until1
# Lee de la línea de comando el login name de un usuario y revisa cada 10 s si ingreso al sistema.
# Cuando entra se termina el ciclo y suena bocina.
#!/bin/bash
until who | grep "$1" > /dev/null
do
    echo "Esperando a $1"
    sleep 10
done

echo -e \\\a
echo "Ya entro $1"
```

DESARROLLO DE LA PRACTICA

1.- Escriba un archivo que contenga los nombres de 5 de sus compañeros. Por ejemplo,

Pedro
Juan
caleb
hkBoy241
Joshua

Grabarlo con el nombre "stalker"

2.-Escriba un script que verifique si los nombres del archivo stalker están conectados.

3.-Escriba un script que reciba de la línea de comando el nombre, apellido paterno y apellido materno e imprima el mensaje "Hola como estas + <nombre>", si el usuario no da los tres parámetros mandar un mensaje de nombre incompleto.

4.- Escriba un script que permita adivinar la capital de Bulgaria, el programa deberá terminar cuando el usuario escriba la capital correcta y muestre la cantidad de intentos realizados.

5.-Escriba un script que lea la fecha de nacimiento de una persona en formato dd/mm/aaaa, e imprima el mes del año que corresponda. Si el usuario no escribe un mes entre 1 y 12 mandar un mensaje de error. Nota: utilice el comando cal.

Si el usuario escribe 16/10/1980 , su script imprimirá:

October 1980						
Su	Mo	Tu	We	Th	Fr	Sa
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	